and does not need to be repeated here. Objects can also be cast to strings provided they implement a magic `__toString()` method; see Chapter 6, "Object Fundamentals," for examples.

Concatenation itself will trigger type coercion (to a string)…

```
var_dump("122" . 1);
Output:
string(4) "1221"
```

… and mathematical operations will coerce to a number:

```
var_dump("122" + 1);
var_dump("124" - 0.6);

Output:
int(123)
float(123.4)
```

Safety

PHP offers many (oh so many) functions for string manipulation (or mangling, if you prefer). Before we get started, there are two very important concepts to be aware of: *binary safety* and *multi-byte safety*.

Binary Safety

As we now know, PHP strings aren't really text; they are buffers containing bytes and metadata indicating the length of the buffer. Some languages use a null byte – *0x00* – as a string terminator (e.g., the language that the PHP interpreter is written in: C); PHP generally does not (it uses the length metadata). Binary safety is a property of PHP functions: a binary unsafe function would treat a null byte as a terminator and potentially truncate the buffer once it encountered the first *0x00* byte value. For example, a buffer might contain the data for an image with potentially hundreds of null bytes and would not survive an encounter with a binary unsafe function: in such a scenario the resulting data output by the function would no longer be an intact image.

Therefore, a binary safe function is one that does not ascribe any special meaning to a null byte, and the function will act on the entire buffer. Most PHP string functions